Lab 12: Social and Media Platform APIs

INFO 4617/5617 · Week 12 · Chapter 12, Social and Media Platform APIs

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll read what federal agencies post on two platforms:

- **Part 1, Monday: Bluesky**, whose public API answers anyone, with no account or key, and how to tell an agency's account from an impostor's;
- **Part 2, Wednesday:** more of Bluesky: more posts than one request returns, and what an agency's followers and reposts can show;
- **Part 3, Friday: YouTube**'s Data API, which needs a key, for the same agencies' videos.

Reddit and Spotify are in the chapter, as platforms that have closed much of what their APIs once offered.

Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Monday and Wednesday need no key. **For Friday, get a YouTube Data API key** from a personal Google account (how), and store it as the environment variable `YOUTUBE_KEY`, one of week 11's three safe ways: set it, close the terminal, open a new one, and start Jupyter from there.

On Wednesday and Friday, start at that day's setup step, or run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Part 1 written and run live on 2026-11-02, the morning of its class, before chapter 12's revision. It reads Bluesky's public AppView with `requests` and no login, which is where the chapter is going (the sprint's plan); the chapter on `main` (textbook commit `baf0bc0`) still uses `atproto` with an app password. So, as week 11's Part 1 did, this part is revised once the chapter's revision merges (`labs/AUTHORING.md`, "After the chapter, when it changes"). Every cell ran with `labs/tools/run_key.py`; no key is needed. Follower and post counts change daily, and the feeds return the newest posts, so a rerun gives different rows. The five accounts are the federal agencies with a `.gov` handle found on 2026-11-02 (NASA, the Census Bureau, USGS and others have none). Reposts are skipped so that no private person's post or handle appears in an output. Parts 2 and 3 are added before their days.

Part 1 — Monday: Bluesky, and who is posting

Bluesky is a social network built on an open protocol, the AT Protocol. Its public API answers anyone who asks, with no account and no key, the way Wikipedia's did in week 10. We start from an agency's profile page on the website, ask the API for the same profile, then compare the federal agencies that post there, and read what one of them posted this week.

Step 0 — Set up (5 minutes)

Import what we need and define the User-Agent: Bluesky asks for no key, but every request still says who sent it, as Wikipedia's did. Every request today goes to Bluesky's public **AppView**, `https://public.api.bsky.app/xrpc/`, the service that answers reads for anyone (API hosts). Each method's name is part of the address: `app.bsky.actor.getProfile` reads a profile, `app.bsky.feed.getAuthorFeed` an account's posts.

```
# Import requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define HEADERS, whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)", with your
 ↪  email
# bsky_base = "https://public.api.bsky.app/xrpc"
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
bsky_base = "https://public.api.bsky.app/xrpc"
```

💬 Discuss

- Census and FRED wanted a key; Bluesky doesn't. What does a key let an agency do that Bluesky gives up?
- Bluesky's protocol lets anyone run an AppView of their own. What would it mean for a researcher if the company's went away?

Step 1 — From the website to the API: one agency's profile (8 minutes)

Open the National Weather Service's profile on the website, bsky.app/profile/nws.noaa.gov. Its **handle**, `nws.noaa.gov`, is a domain name: an account can take a domain as its handle only by proving it controls that domain (Bluesky's guide), so no one but NOAA can post as `nws.noaa.gov`. The blue check beside its name is a second proof: Bluesky itself has **verified** the account (how verification works). Now ask the API for the same page with `app.bsky.actor.getProfile`, and find the website's numbers in its answer, as the chapter's "Bluesky via AT Protocol" reads a profile.

```
# profile: the JSON of f"{bsky_base}/app.bsky.actor.getProfile" with params={"actor":
 ↪  "nws.noaa.gov"} and HEADERS
# Print its handle, displayName, followersCount, postsCount, and createdAt; then its
 ↪  "verification"
profile = requests.get(f"{bsky_base}/app.bsky.actor.getProfile", params={"actor":
 ↪  "nws.noaa.gov"}, headers=HEADERS).json()
print(profile["handle"], "|", profile["displayName"])
print("followers:", profile["followersCount"], "| posts:", profile["postsCount"], "| joined:",
 ↪  profile["createdAt"][:10])
print(profile["verification"]["verifiedStatus"], "by",
 ↪  profile["verification"]["verifications"][0]["issuerHandle"])
```

```
nws.noaa.gov | National Weather Service
followers: 63020 | posts: 1394 | joined: 2025-01-17
valid by bsky.app
```

💬 Discuss

- Did the API's counts match the website's? If they differ by a few, why might that be?
- A search for "National Park Service" on Bluesky finds "Alt National Park Service" and "The National Park Service UNCENSORED", with no `.gov` handle and no check. What would a researcher who collected their posts as the agency's have got wrong?
- Which proof would you trust more, the domain or the check, and why?

Step 2 — Five agencies in one table (8 minutes)

Five federal accounts have a `.gov` handle on Bluesky as of November 2026; four of them are NOAA's. Ask for each profile in a loop, keep the same five fields, and make a DataFrame, sorted by followers, as week 11 compared places. `app.bsky.actor.getProfiles` can ask for up to 25 at once; one at a time keeps the loop plain.

```
# agencies: "noaa.gov", "nws.noaa.gov", "climate.noaa.gov", "noaacomms.noaa.gov",
 ↪  "federalreserve.gov"
# rows: for each handle, request its profile, and append a dict of handle, displayName,
 ↪  followersCount, postsCount, createdAt[:10]
# accounts: a DataFrame of rows, sorted by followersCount, largest first; show it
agencies = ["noaa.gov", "nws.noaa.gov", "climate.noaa.gov", "noaacomms.noaa.gov",
 ↪  "federalreserve.gov"]
rows = []
for handle in agencies:
    p = requests.get(f"{bsky_base}/app.bsky.actor.getProfile", params={"actor": handle},
 ↪  headers=HEADERS).json()
    rows.append({"handle": p["handle"], "name": p["displayName"], "followers":
 ↪  p["followersCount"],
                 "posts": p["postsCount"], "joined": p["createdAt"][:10]})
accounts = pd.DataFrame(rows).sort_values("followers", ascending=False)
accounts
```

	handle	name	followers	posts	joined
1	nws.noaa.gov	National Weather Service	63020	1394	2025-01-17
2	climate.noaa.gov	NOAA Climate.gov	46966	93	2025-01-14
0	noaa.gov	NOAA	41937	150	2025-01-14
4	federalreserve.gov	Federal Reserve	8405	1029	2024-12-06
3	noaacomms.noaa.gov	NOAA Communications	5620	117	2025-01-14

⏱️ Microsprint (3 minutes, alone or in pairs)

Not every official account has a domain handle. The Library of Congress posts as `librarycongress.bsky.social`, and the Federal Reserve Bank of St. Louis, FRED's home in week 11, as `stlouisfed.bsky.social`.

Do: add both to the loop, and print each account's `verification` status. **Share:** which of the seven are verified, and how you'd tell an official `.bsky.social` account from an impostor's.

```
# Microsprint: for "librarycongress.bsky.social" and "stlouisfed.bsky.social", request the
 ↪  profile
# Print each handle, its followersCount, and its verification's verifiedStatus
for handle in ["librarycongress.bsky.social", "stlouisfed.bsky.social"]:
    p = requests.get(f"{bsky_base}/app.bsky.actor.getProfile", params={"actor": handle},
 ↪  headers=HEADERS).json()
    print(p["handle"], "|", p["followersCount"], "|", p["verification"]["verifiedStatus"])
```

```
librarycongress.bsky.social | 9237 | valid
stlouisfed.bsky.social | 1121 | valid
```

💬 Discuss

- Most federal agencies, the Census Bureau and NASA among them, have no `.gov` handle on Bluesky. What does that say about which agencies a study of "government on Bluesky" would see?
- The National Weather Service has more followers than NOAA, its parent. What might explain it?
- The Federal Reserve joined first but has the fewest followers. Is followers the right measure of an agency's reach?

Step 3 — What an agency posted this week (10 minutes)

`app.bsky.feed.getAuthorFeed` returns an account's posts, newest first, up to 100 a request. With `filter="posts_no_replies"` it leaves out the account's replies, but it still includes its **reposts**: other

people's posts it shared, which carry a `"reason"`. Those authors may be private people, so skip any item with a reason, and keep each post's time, its counts, and the start of its text.

```
# feed: the JSON's "feed" of f"{bsky_base}/app.bsky.feed.getAuthorFeed", params
 ↪  actor="nws.noaa.gov", limit=25, filter="posts_no_replies"
# posts: for each item without a "reason" (not a repost), a dict of its post's indexedAt[:16],
 ↪  likeCount, repostCount, replyCount, and record text[:70]
# nws_posts: a DataFrame of posts; show it sorted by repostCount, largest first, the top 5
params = {"actor": "nws.noaa.gov", "limit": 25, "filter": "posts_no_replies"}
feed = requests.get(f"{bsky_base}/app.bsky.feed.getAuthorFeed", params=params,
 ↪  headers=HEADERS).json()["feed"]
posts = []
for item in feed:
    if "reason" in item:
        continue
    post = item["post"]
    posts.append({"time": post["indexedAt"][:16], "likes": post["likeCount"], "reposts":
 ↪  post["repostCount"],
                  "replies": post["replyCount"], "text": post["record"]["text"][:70]})
nws_posts = pd.DataFrame(posts)
print(len(feed), "items,", len(nws_posts), "of them the agency's own posts")
nws_posts.sort_values("reposts", ascending=False).head(5)
```

```
25 items, 25 of them the agency's own posts
```

	time	likes	reposts	replies	text
0	2026-10-31T18:01	206	80	1	Daylight Saving Time ends at 2am overnight ton...
1	2026-10-30T20:59	89	33	3	Updated Extreme Rainfall Outlook for Sunday:\n...
23	2026-10-20T16:10	118	33	3	Here's a more detailed look at the expected ra...
20	2026-10-21T19:55	120	32	1	A subtropical storm could form later today or ...
21	2026-10-21T14:27	90	31	0	A rare risk for tornadoes and severe weather e...

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: run the same request for another of the five agencies. **Share:** the agency, how many of its 25 items were its own posts rather than reposts, and the topic of its most-reposted post.

```
# Microsprint: set params["actor"] to another agency; request again, build the DataFrame the same
 ↪  way, and show its top 3 by reposts
params["actor"] = "federalreserve.gov"
feed = requests.get(f"{bsky_base}/app.bsky.feed.getAuthorFeed", params=params,
 ↪  headers=HEADERS).json()["feed"]
posts = []
for item in feed:
    if "reason" in item:
        continue
    post = item["post"]
    posts.append({"time": post["indexedAt"][:16], "likes": post["likeCount"], "reposts":
 ↪  post["repostCount"],
                  "replies": post["replyCount"], "text": post["record"]["text"][:70]})
fed_posts = pd.DataFrame(posts)
```

```
print(len(feed), "items,", len(fed_posts), "of them the agency's own posts")
fed_posts.sort_values("reposts", ascending=False).head(3)
```

```
25 items, 25 of them the agency's own posts
```

	time	likes	reposts	replies	text
7	2026-10-28T20:30	5	5	0	Watch Chairman Warsh’s statement from the #FOM...
12	2026-10-28T14:22	1	4	0	WATCH LIVE TODAY: Press conference with #FOMC ...
13	2026-10-27T14:07	6	3	2	Tomorrow at 2:30 p.m. ET: Chairman Warsh hosts...

💬 **Discuss**

- Which kinds of posts got reposted most: warnings, explainers, or announcements? What would you need to say a kind of post causes reposts?
- We skipped reposts to keep private people's posts out of our table. What does an agency's reposting tell you that its own posts don't, and how could you study it without printing anyone's handle?
- The feed's last item came with a `cursor`. What do you think it's for? (Week 10's `continue` is a hint.)

End of Part 1

On Wednesday, Part 2 follows the `cursor` to read more than one request returns, and asks what an agency's followers and posting rhythm can show. It starts with its own setup step, so you don't need today's cells to have run.

Before Friday: get your YouTube Data API key, from a personal Google account, and store it as `YOUTUBE_KEY`.